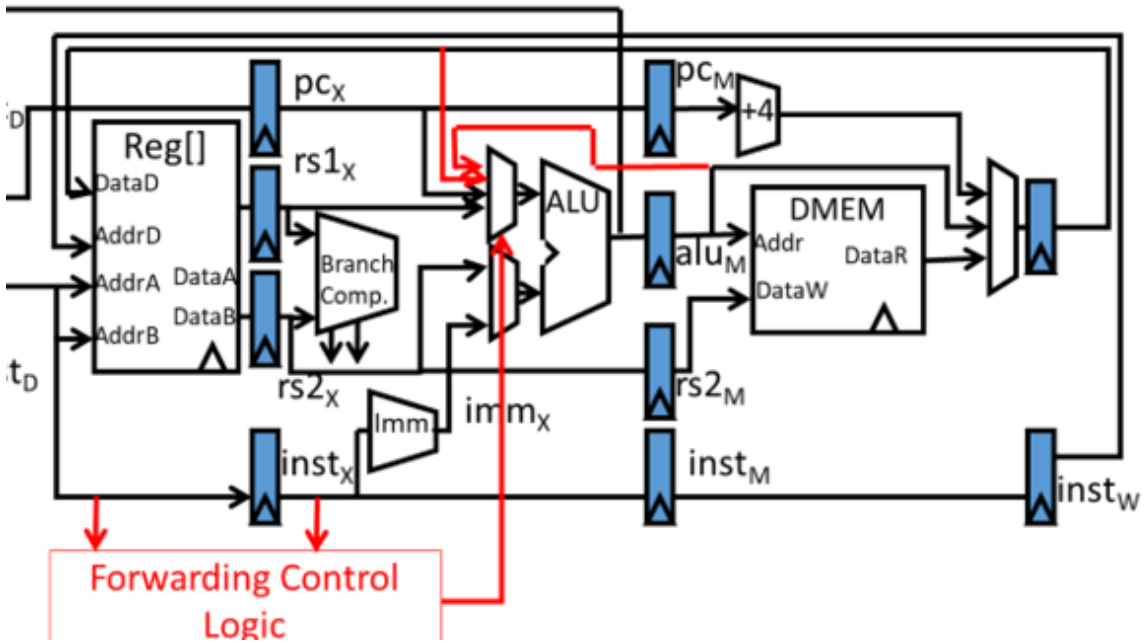
Main Features Of CPU

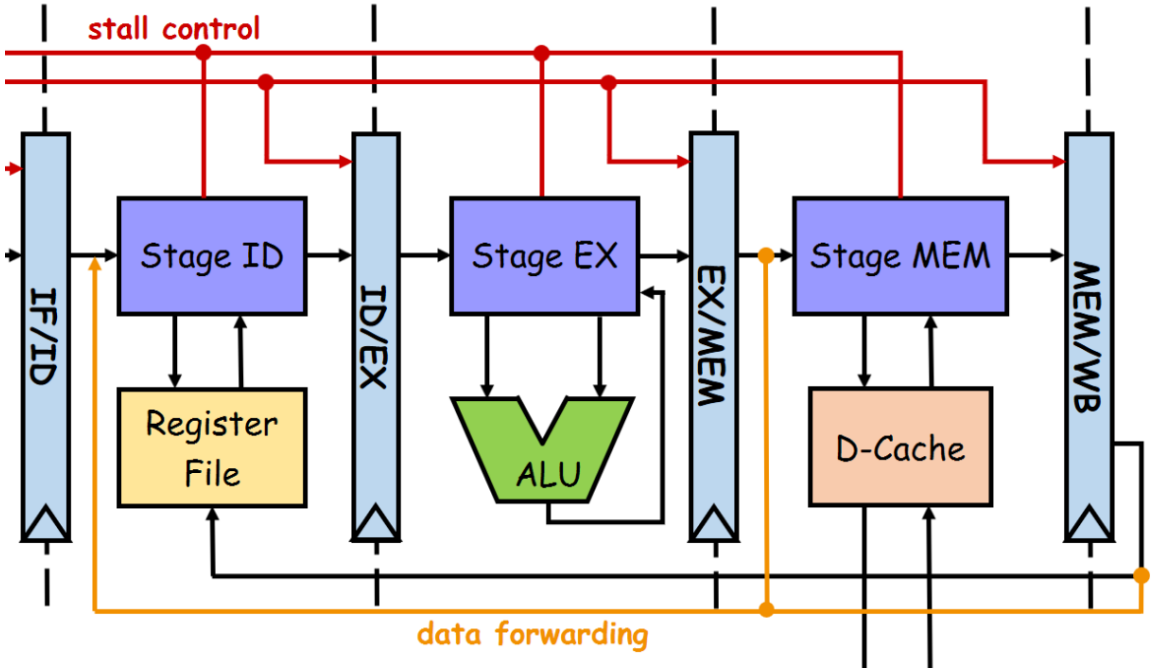
- ISA:RISC-V (RV32I subset)
- 5-stage Pipelining
- Complete forwarding path
- Bus set associate DMEM,Ip core block ROM as IMEM
- Passed bubble sort test
- UART module passed simulation independently

01 Pipeline-data forwarding

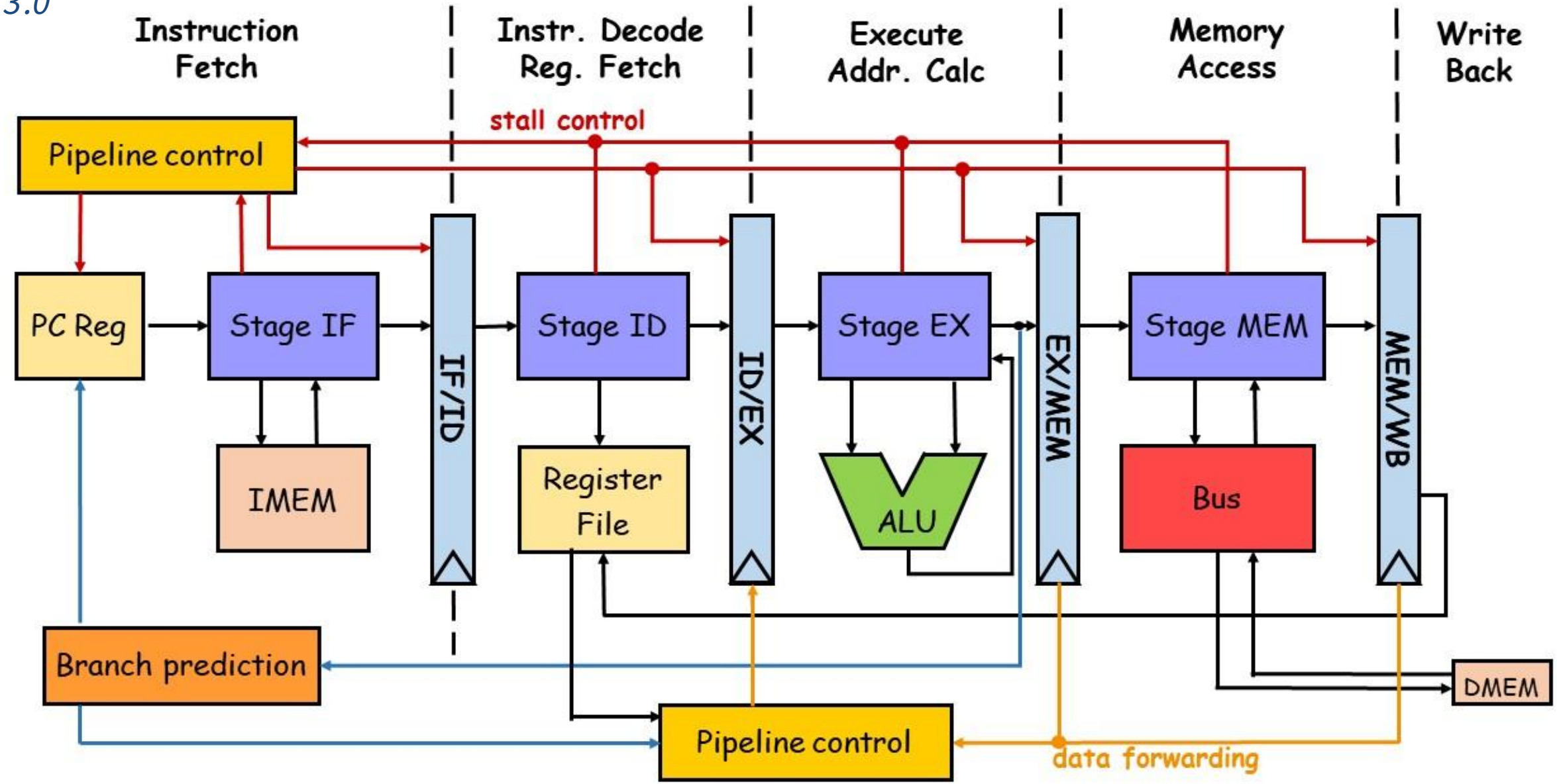
◆ *V1.0*



◆ *V2.0*



◆ *V3.0*

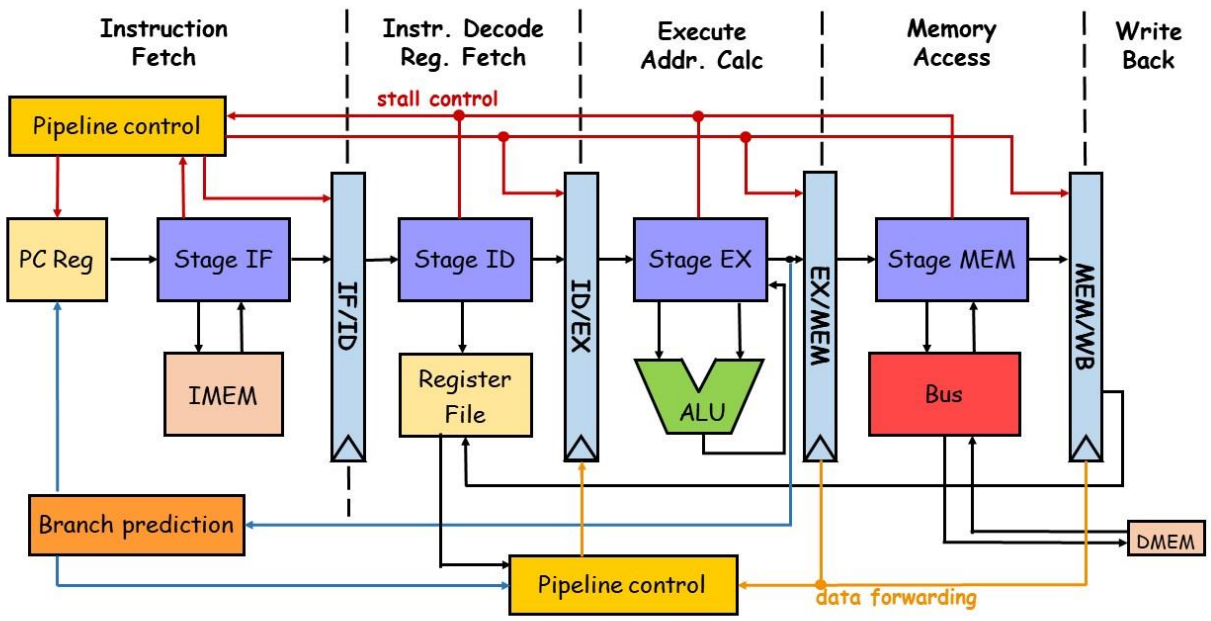


5-Stage Pipeline of RISC-V CPU by our team

01 Forwarding Data Priority

◆ Analysis

ex>mem>**wb>reg[]**



5-Stage Pipeline of RISC-V CPU by our team

◆ Code

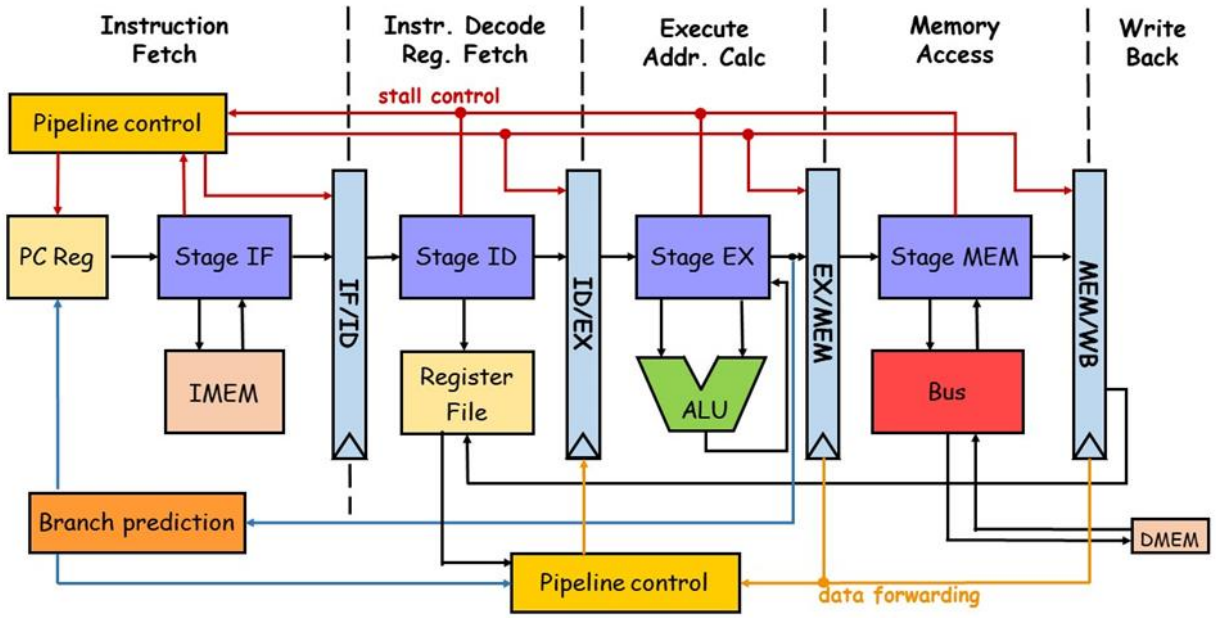
reg.v

```
/* read register A*/
always @(*) begin
 if( RegWen && ( raddrA_i == waddrD_i) )begin
        rdataA_o = wdataD_i;
 end else begin//read reg normally
        rdataA_o = regs[raddrA_i];
 end
end
```

01 Forwarding Data Priority

◆ Analysis

ex>mem>wb>reg[]



5-Stage Pipeline of RISC-V CPU by our team

◆ Code

Forw_ctrl.v

```
/* forwarding dataA control */
always@(*) begin
    if( ex_rd && ( ex_we ) && ( rs1 == ex_rd ) )begin
        forw_dataA_o=ex_alu_i;
    end else if( mem_rd && ( mem_we )
                    && ( rs1==mem_rd ) )begin
        forw_dataA_o=mem_31sel_i;
    end else begin
        forw_dataA_o=reg_dataA_i;
    end
end
```

- Method 1

Resources	time	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
	IF	I_1	I_2	I_3	I_4	I_5				
	ID		I_1	I_2	I_3	I_4	I_5			
	EX			I_1	I_2	I_3	I_4	I_5		
	MA				I_1	I_2	I_3	I_4	I_5	
	WB					I_1	I_2	I_3	I_4	I_5

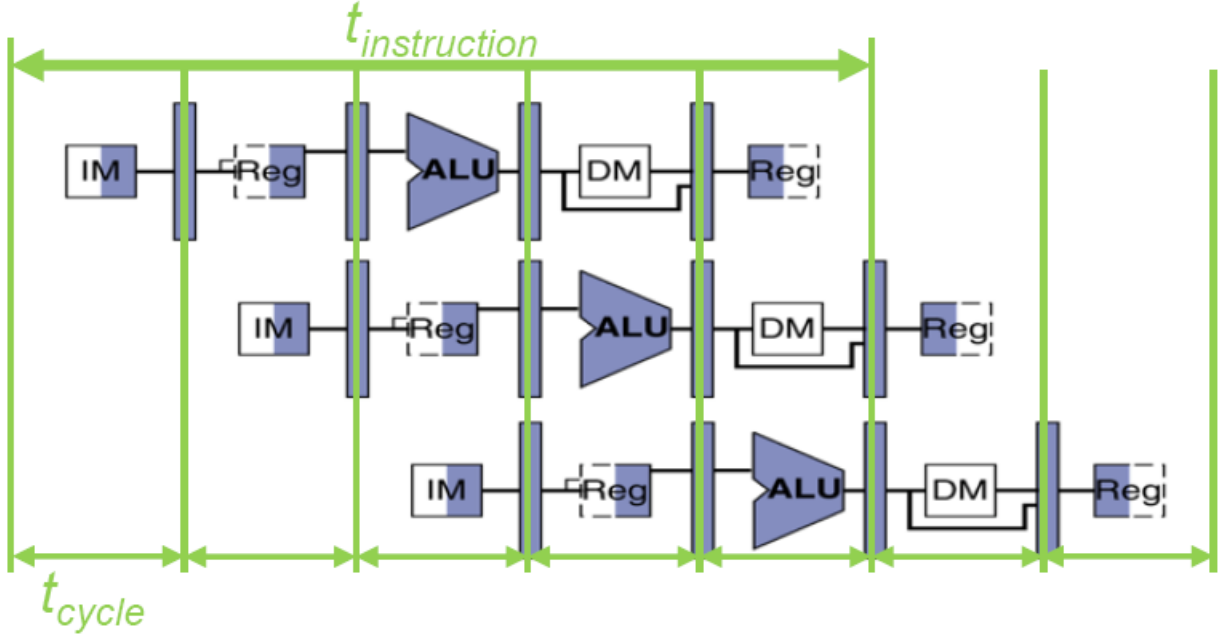
- Method 2

instruction sequence

add t0, t1, t2

or t3, t4, t5

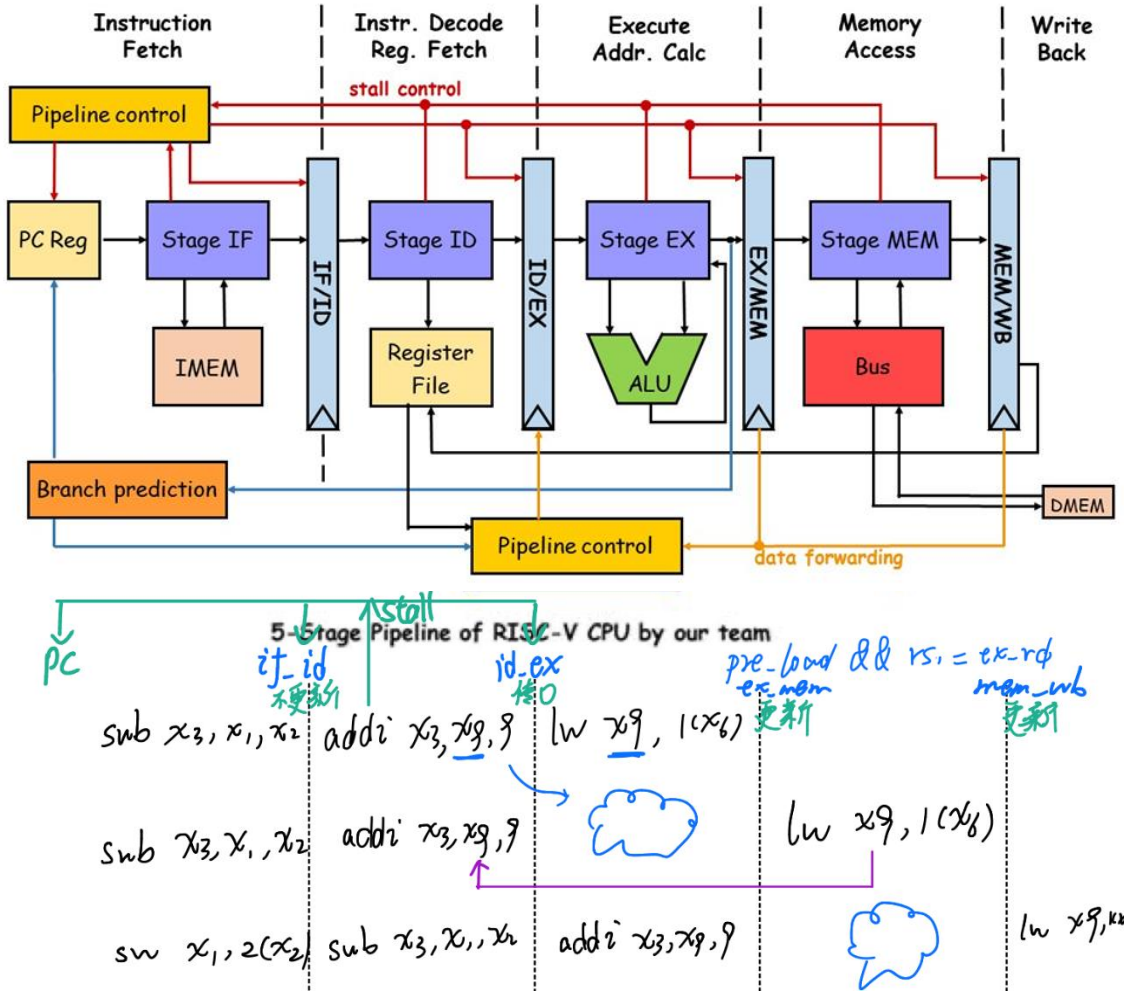
sll t6, t0, t3



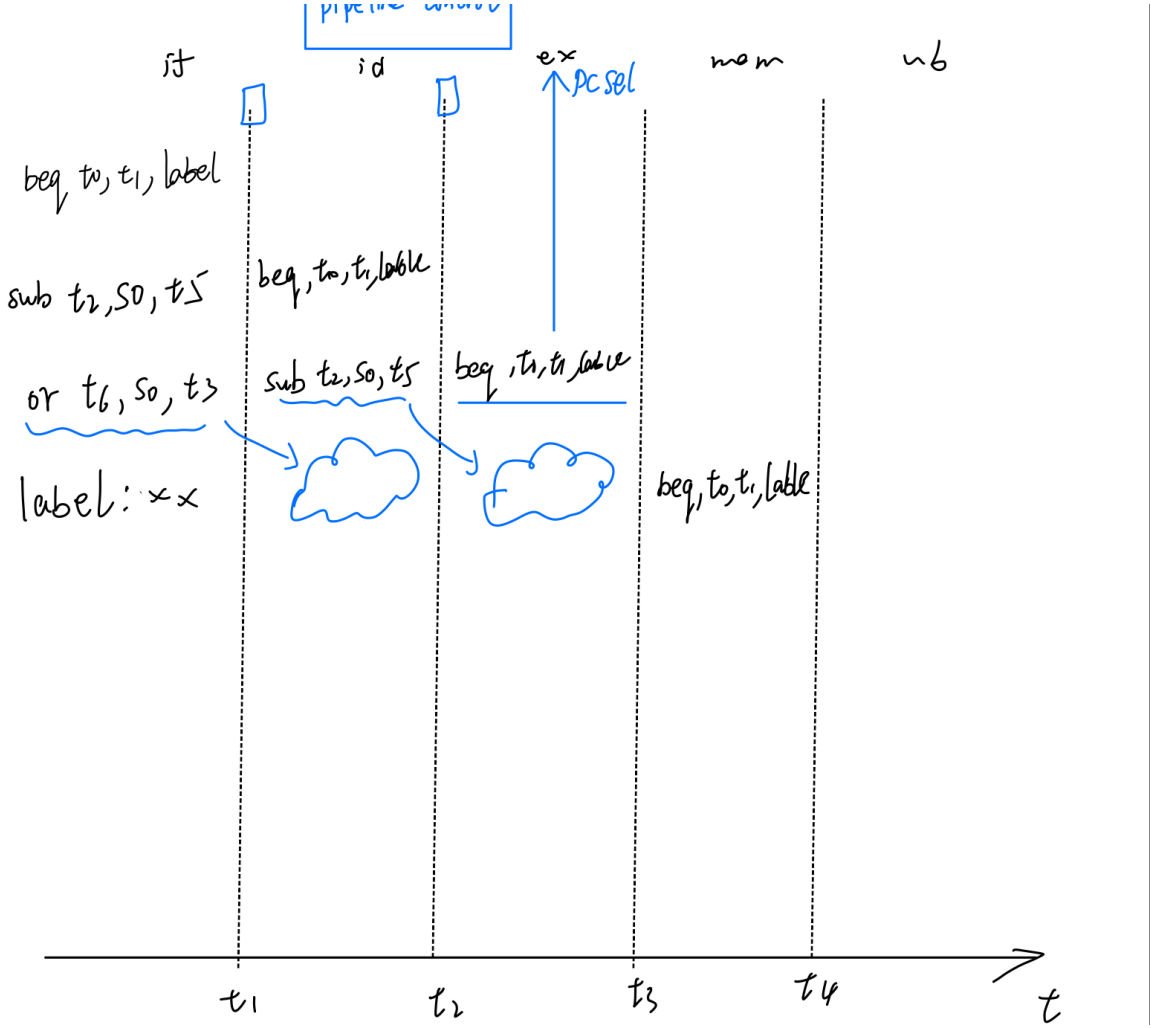
02 Stall control

My analysis method

◆ Load



◆ branch



pipeline reg has 3 status

02 Stall control

Code

◆ Forw_ctrl.v

```
/* stall control */
assign stage_id_stallreq=
(ex_is_load && (rs1 == ex_rd  ||  rs2
== ex_rd )) ;
always@(*) begin
    if(!rst) begin
        stall <= 6'b000000;
    end else
if( stage_id_stallreq ) begin
stall <= 6'b000111;
end else begin
        stall <= 6'b000000;
    end
end
```

! load

◆ If_id.v

```
/* reg_if_id module */
  always @ ( posedge clk ) begin
if( !rst || br ||(stall[1] && !stall[2]))
begin
        id_pc_o <= 32'd0;
        id_inst_o <= 32'd0;
    end
    else if( !stall[1] ) begin
        id_pc_o <= pc_i;
        id_inst_o <= inst_i;
    end
 end
```

- Not updated

◆ Id_ex.v

```
/* reg_id_ex module */
  always @ ( posedge clk ) begin
    if( !rst ||  br ||(stall[2]
&& !stall[3])) begin
        ex_pc_o <= 0;
        ex_inst_o <= 0;
  end else if( !stall[2] )
begin
        ex_pc_o  <= id_pc_i;
        ex_opv1_o <= id_opv1_i;
 end
end
```

- Ouput “0”

02 Stall control

Code

◆ Branch_comp.v

```
/* Branch Comparator */

assign PCSel   = (i_beq & BrEQ)
| (i_bne & ~BrEQ) | (i_blt &
BrLT)
          | (i_bltu & BrLT)|
(i_bge & ~BrLT)| (i_bgeu &
~BrLT)
          | i_jal | i_jalr;
```

! br

◆ If_id.v

```
/* reg_if_id module */
  always @ ( posedge clk ) begin
if( !rst || br ||(stall[1] && !stall[2]))
begin
      id_pc_o <= 32'd0;
      id_inst_o <= 32'd0;
    end
    else if( !stall[1] ) begin
      id_pc_o <= pc_i;
      id_inst_o <= inst_i;
    end
 end
```

- Ouput “0”

◆ Id_ex.v

```
/* reg_id_ex module */
  always @ ( posedge clk ) begin
    if( !rst ||  br ||(stall[2]
&& !stall[3])) begin
      ex_pc_o <= 0;
      ex_inst_o <= 0;
  end else if( !stall[2] )
begin
      ex_pc_o  <= id_pc_i;
      ex_opv1_o <= id_opv1_i;
 end
end
```

- Ouput “0”

03 5-stage Pipeline cpu test

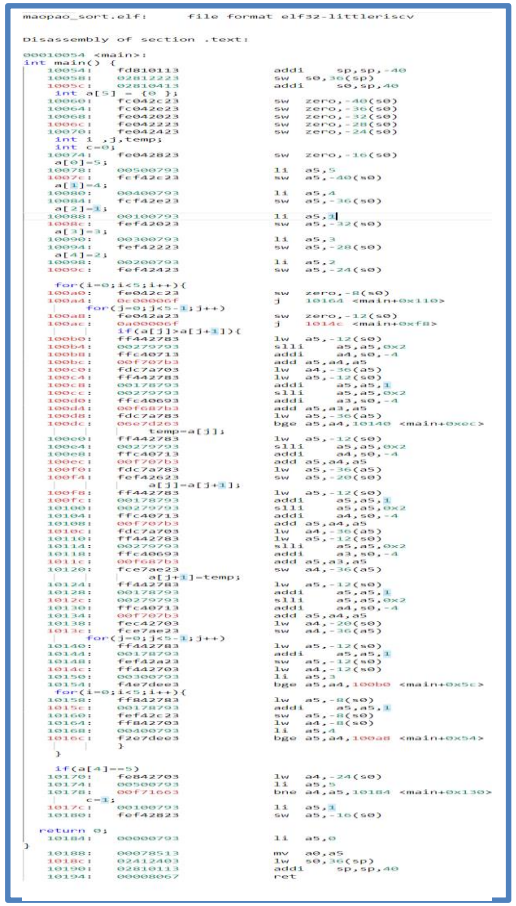
Bubble sort

◆ Bubble_sort.c

```
int main(){
  int a[5]={3,4,5,2,1};
  int i,j,temp;
  for(i=0;i<5;i++){
    for(j=0;j<5-1;j++)
      if(a[j]>a[j+1]){
        temp=a[j];
        a[j]=a[j+1];
        a[j+1]=temp;
      }
    }
  return 0;
}
```

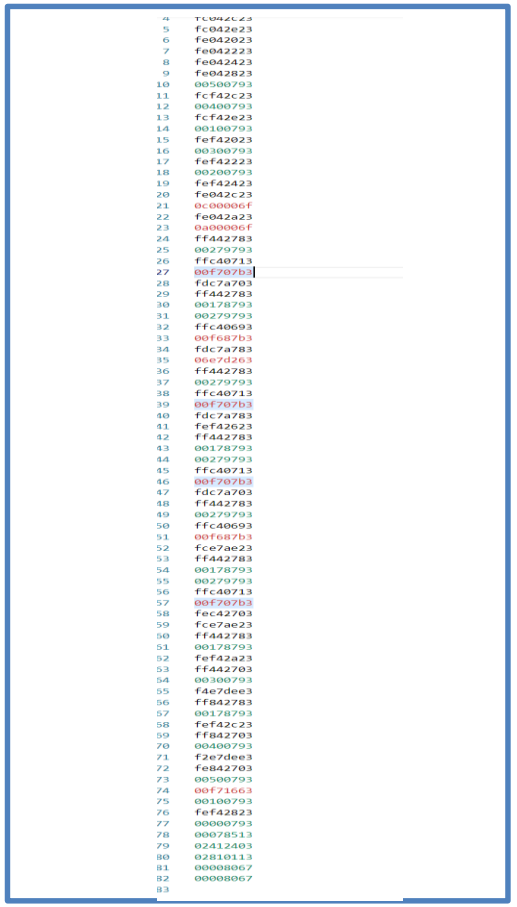
13 lines

◆ Bubble_sort.asm



113 lines

◆ Bubble_sort.dat



83 inst

File Edit Flow Tools Reports Window Layout View Run Help Quick Access Synthesis Out-of-date details

10 ns Default Layout

Flow Navigator

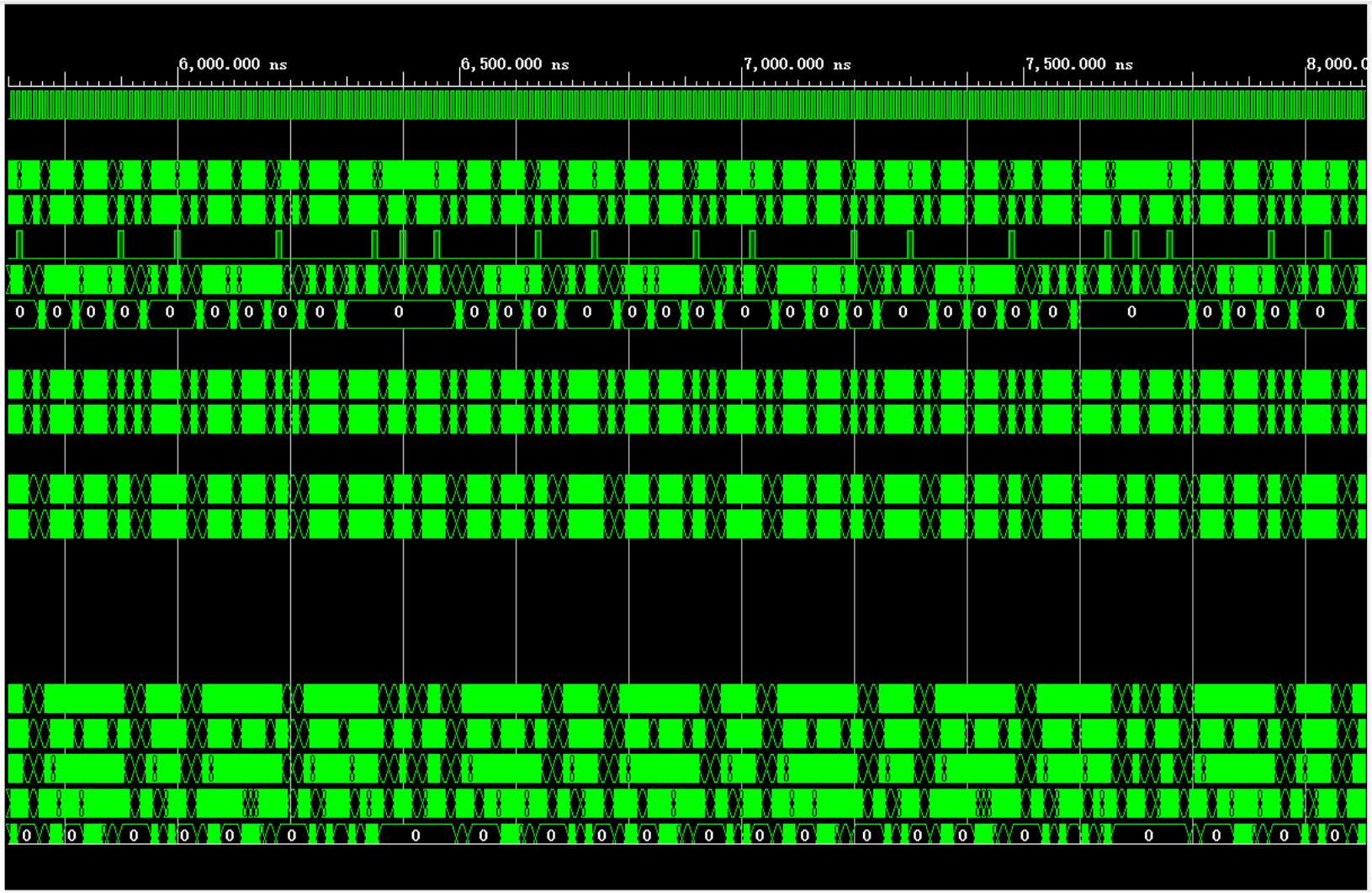
- **PROJECT MANAGER**
  - Settings
  - Add Sources
  - Language Templates
  - IP Catalog
- **IP INTEGRATOR**
  - Create Block Design
  - Open Block Design
  - Generate Block Design
- **SIMULATION**
  - Run Simulation
- **RTL ANALYSIS**
  - Open Elaborated Design
- **SYNTHESIS**
  - Run Synthesis
  - Open Synthesized Design
- **IMPLEMENTATION**
  - Run Implementation
  - Open Implemented Design
- **PROGRAM AND DEBUG**
  - Generate Bitstream

SIMULATION - Behavioral Simulation - Functional - sim_1 - tb

Scope Sources Objects Protocol Instances

top.v | pc_reg.v | **tb_behav.wcfg*** | reg_mem_wb.v | Branch_Comp.v

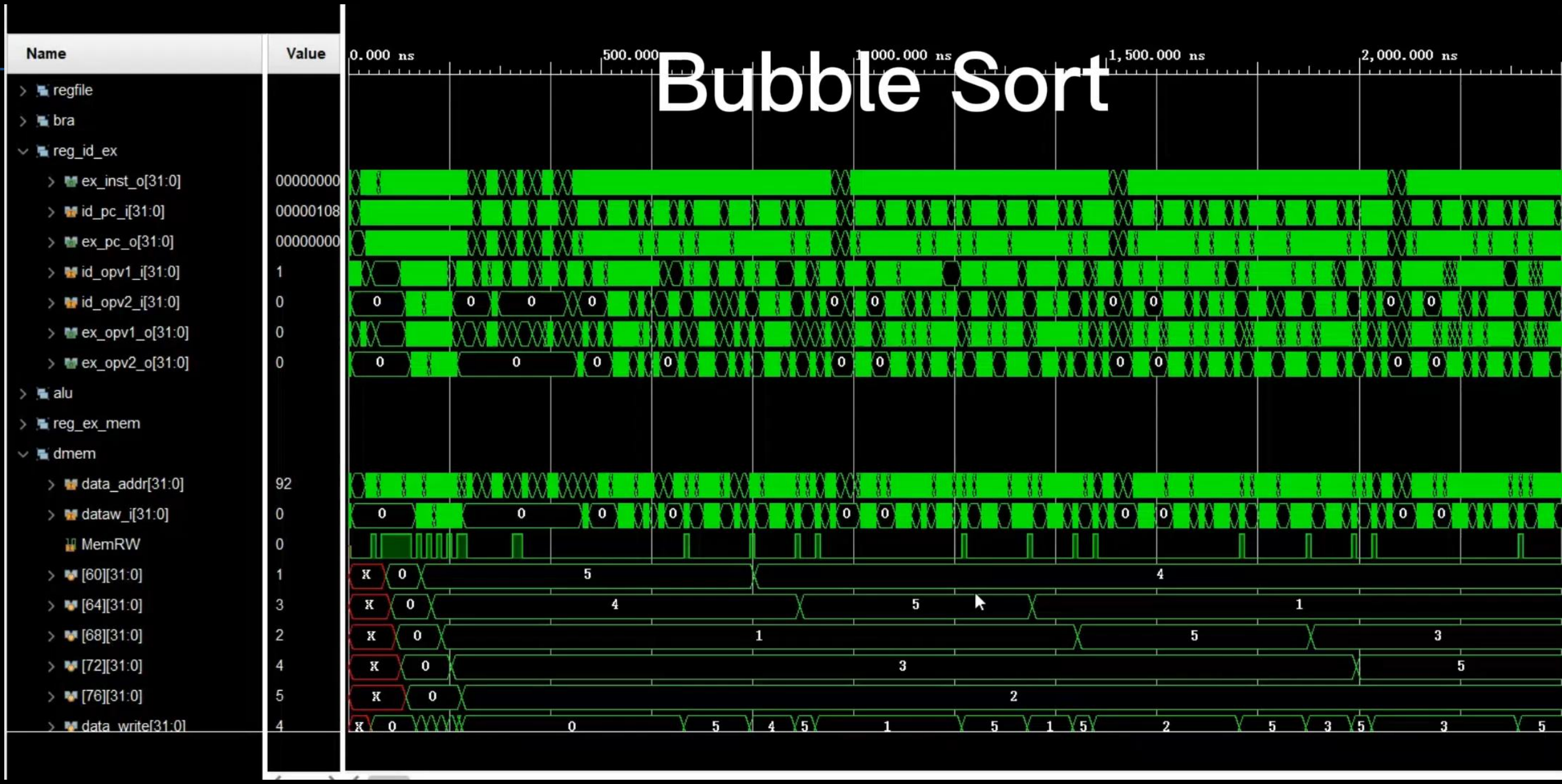
Name	Value
clk	1
pc_reg	
pc[31:0]	00000110
pc_o[31:0]	0000010c
PCSel	0
br_addr_i[31:0]	00000000
stall[5:0]	0
imem	
inst_i[31:0]	fef42c23
inst_addr[31:0]	268
reg_if_id	
id_pc_o[31:0]	00000108
id_inst_o[31:0]	00178793
forw_ctrl	
regfile	
bra	
reg_id_ex	
ex_inst_o[31:0]	00000000
id_pc_i[31:0]	00000108
ex_pc_o[31:0]	00000000
id_opv1_i[31:0]	1
id_opv2_i[31:0]	0



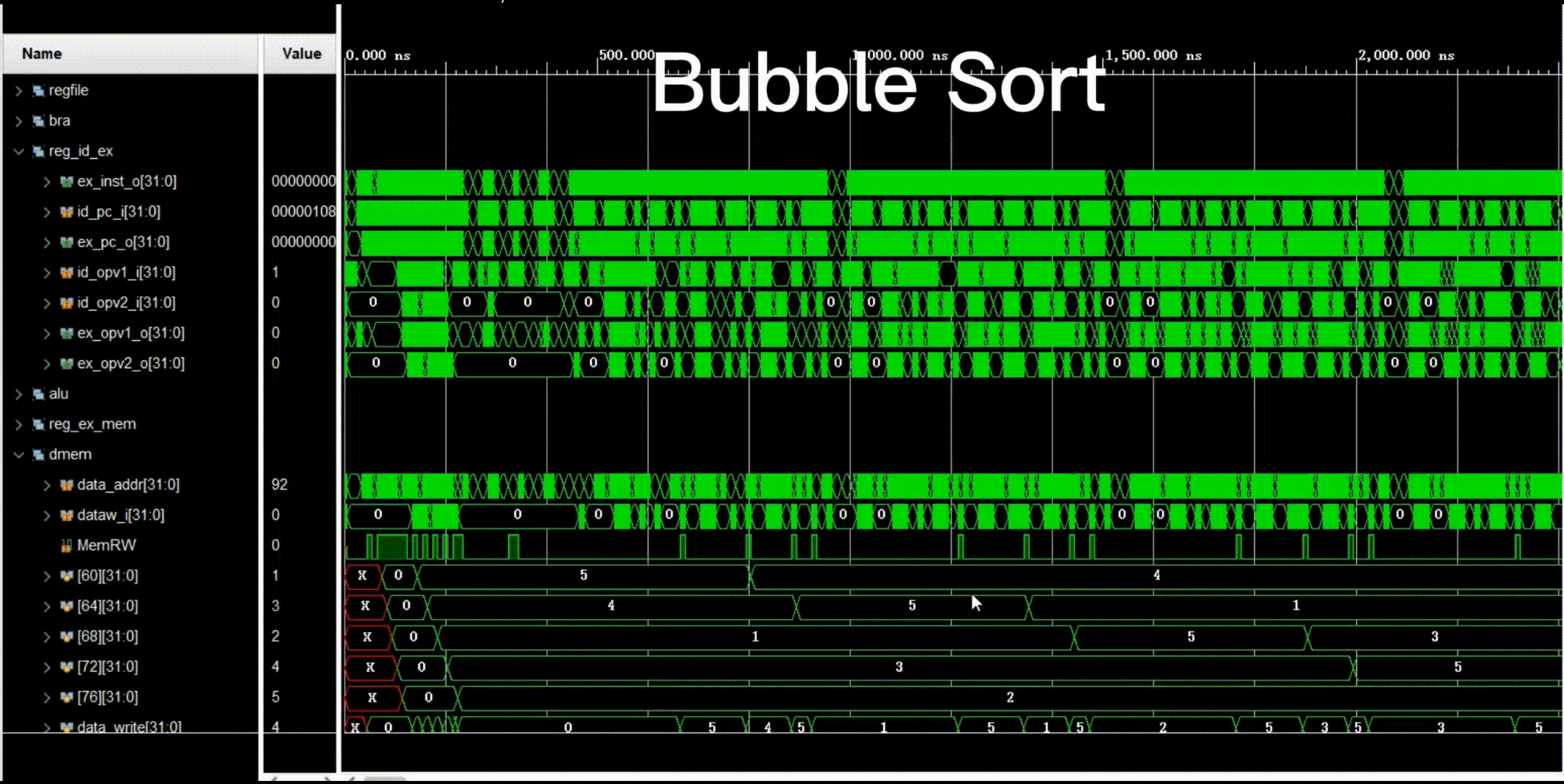
Tcl Console | Messages | Log

Sim Time: 108225 ns

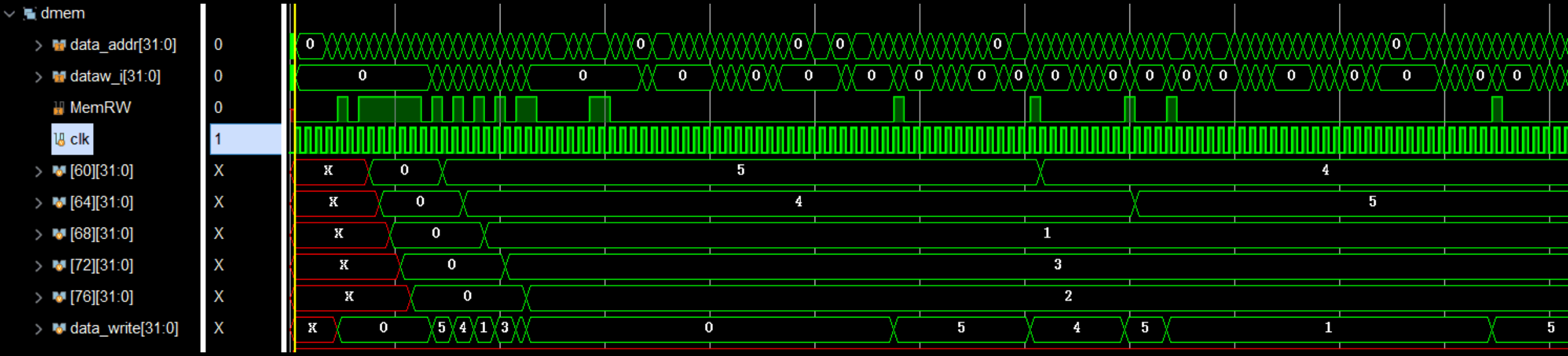
Success!!! Without Branch Prediction ,Cost 567 clks



Success!!! Without Branch Prediction ,Cost 567 clks



With Branch Prediction　Cost 563 clks



03 5-stage Pipeline cpu test

Branch prediction benefit

◆ Without bp

Cost clks: 567

jump times :20

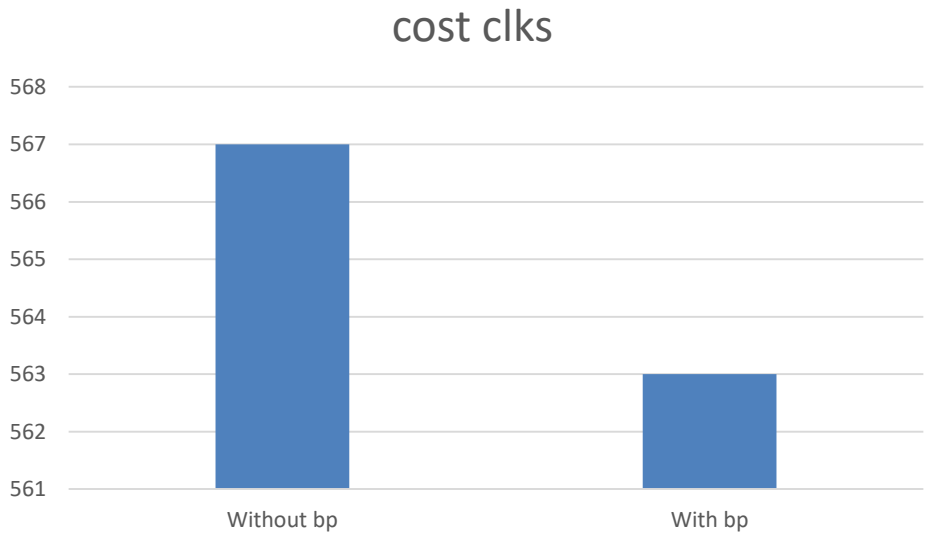
Br bubbles:20x2=40

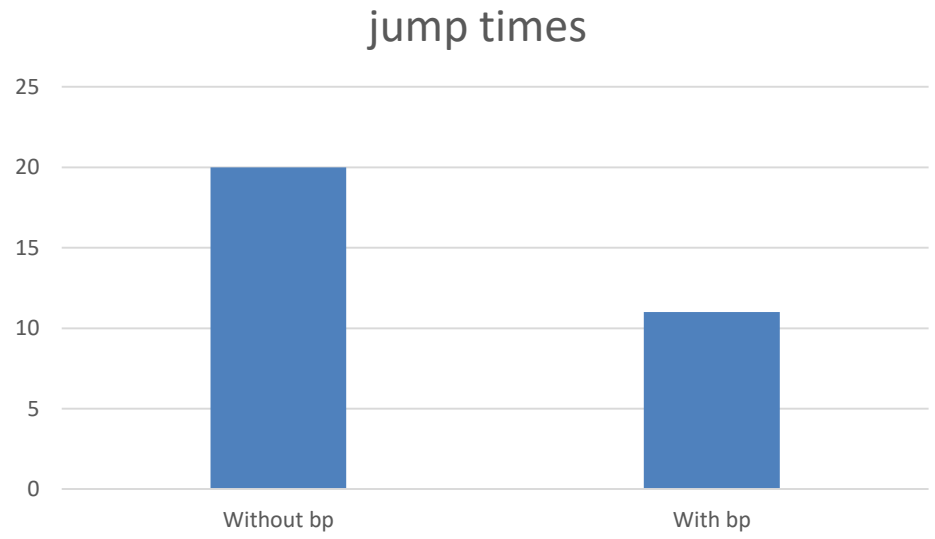
◆ With bp

Cost clks: 563

jump times :11

Bp error penalization:11x3=33





The clks have been reduced by 0.71%

The jump times have been reduced by 45%

03 5-stage Pipeline cpu test

Bubble sort | board test

ricvI32.xdc

```
set_property PACKAGE_PIN U18 [get_ports clk]
set_property PACKAGE_PIN N15 [get_ports rst]
set_property PACKAGE_PIN M14 [get_ports led1]
set_property PACKAGE_PIN M15 [get_ports led2]
set_property PACKAGE_PIN K16 [get_ports led3]
set_property PACKAGE_PIN J16 [get_ports led4]

set_property IOSTANDARD LVCMOS33 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports led1]
set_property IOSTANDARD LVCMOS33 [get_ports led2]
set_property IOSTANDARD LVCMOS33 [get_ports led3]
set_property IOSTANDARD LVCMOS33 [get_ports led4]

create_clock -period 11 -name sysclk [get_ports clk]
```

Top.v

```
assign led1 = `dmem_status_1;
assign led2 = `dmem_status_2;
assign led3 = `dmem_status_3;
assign led4 = `dmem_status_4;
```

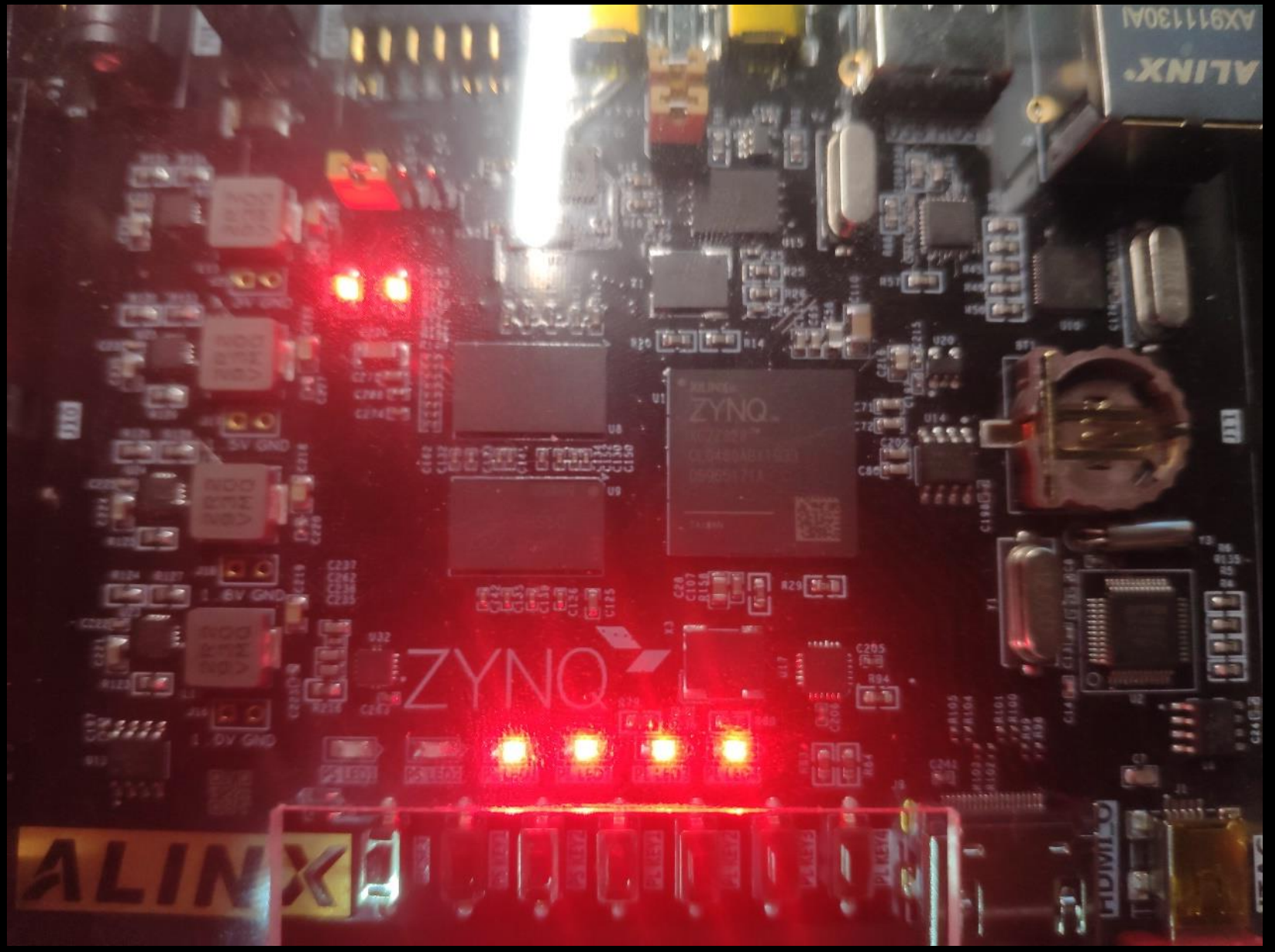
Dmem.v

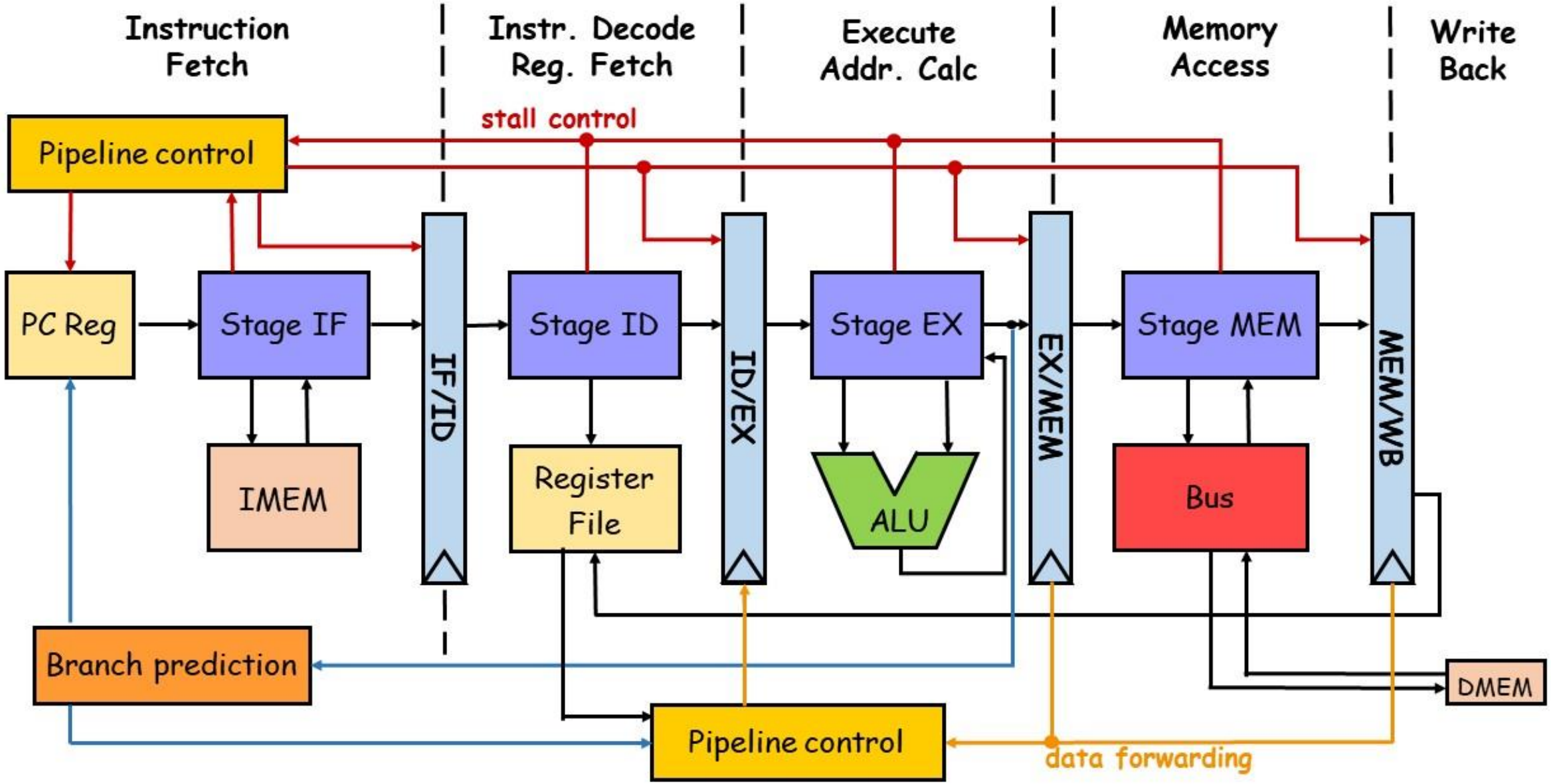
```
    assign status_1 = (_imem[60] == 'd1);
    assign status_2 = (_imem[64] == 'd2);
    assign status_3 = (_imem[68] == 'd3);
    assign status_4 = (_imem[72] == 'd4);
    assign status_5 = (_imem[76] == 'd5);
```

Tb.v

```
//DUV instance
top top_tb(
    .clk      (   clk),
    .rst      (   rst),
    .led1     (   led1),
    .led2     (   led2),
    .led3     (   led3),
    .led4     (   led4)
//    .led5     (   led5)
);
```

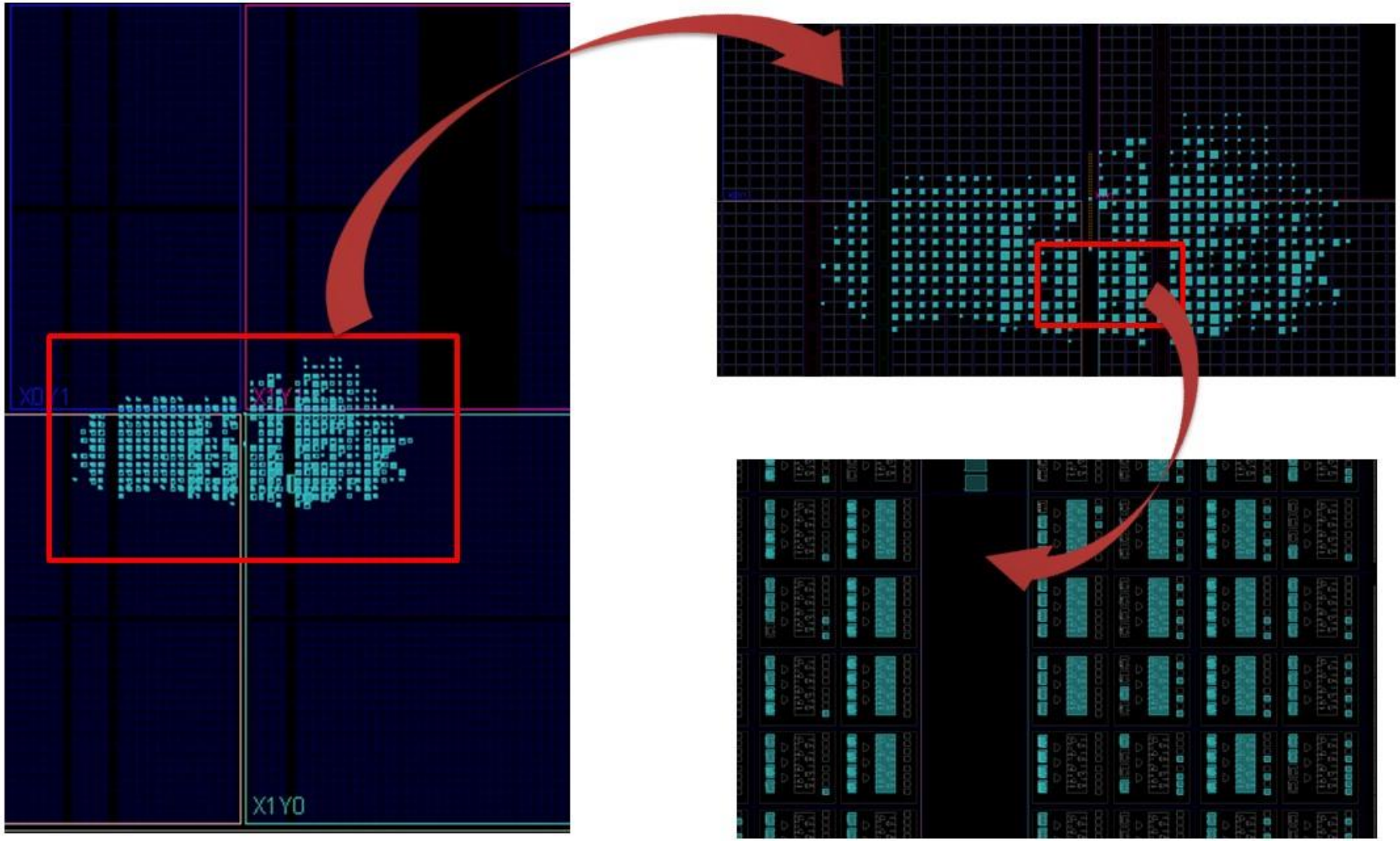
Success!!!



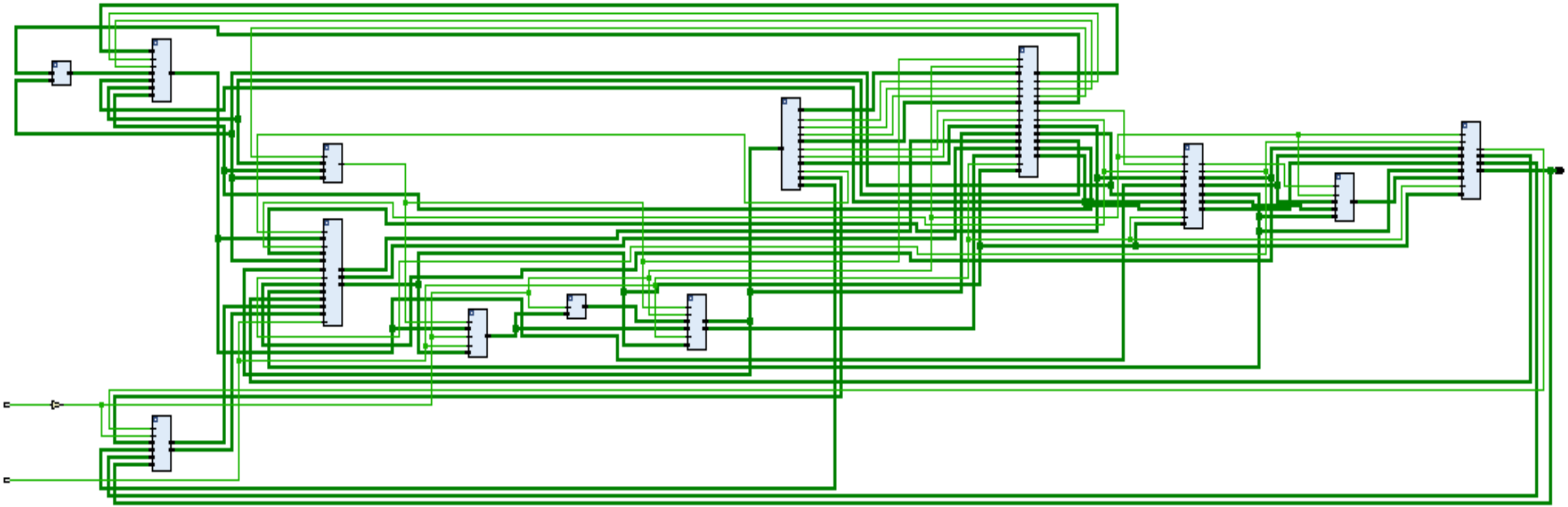


5-Stage Pipeline of RISC-V CPU by our team

Implementation on ZYNQ XC7Z020 FPGA



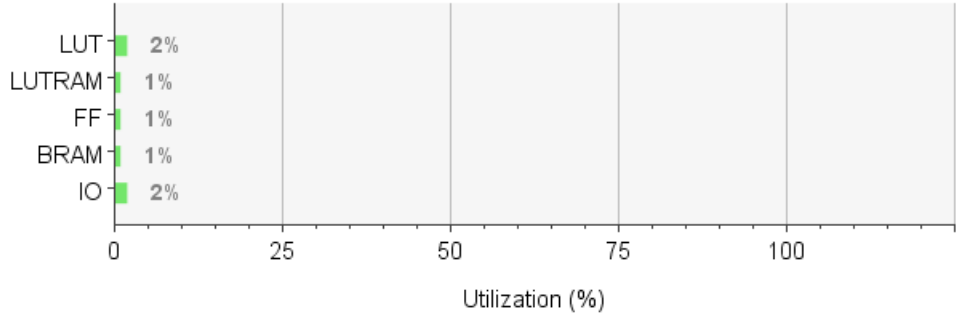
RTL Design



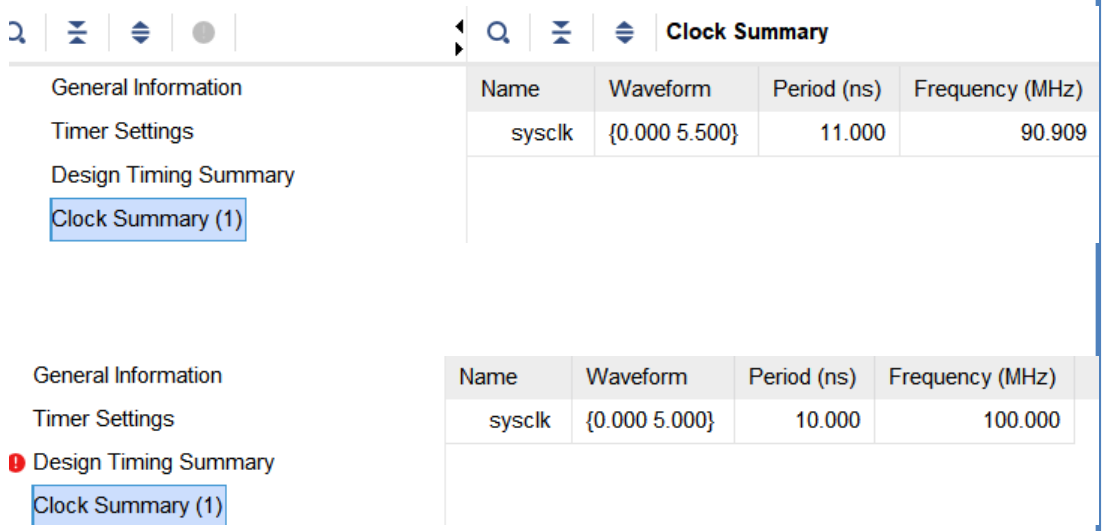
◆ Resource Summary

Summary

Resource	Utilization	Available	Utilization %
LUT	1166	53200	2.19
LUTRAM	112	17400	0.64
FF	517	106400	0.49
BRAM	0.50	140	0.36
IO	3	125	2.40



◆ Clock Frequency : 90.909MHz



04 Summary

◆ References

1. RISC-V-Reader-Chinese-v2p1.pdf
2. https://github.com/Evensgn/RISC-V-CPU byZhou Fan
3. L04 & 05 .pptx
4. tinyriscv: 一个从零开始写的极简、非常易懂的RISC-V处理器核。 - Gitee.com
5. **RISC-V** 基础 | suda-morris 个人博客
6. 《手把手教你设计CPU——RISC-V处理器篇》

◆ Acknowledgements

- Express our appreciation to
  - Mr. Chen and Mr. Wu
  - @Yuanshiwuyu
  - @Utrallyj for the guide of pipeline
  - @cc and @rg for the little core
  - @pupu for the interaction

05 Q & A

◆ Question

- 盯着波形调试精神可嘉，感觉像“愚公移山”，有点费眼睛、废肩膀，请问如何进行更好的调试?
- ~~（老师上周问卷说有同学肩颈不适，建议再出一个问卷调查一下和这门课大作业的相关度）~~

◆ Answer

